

INT202
Complexity of Algorithms
Number Theory and Cryptography

2025-2026

Cryptographic communications

Throughout history there has often been the need (or desire) to *securely* transmit information through *insecure* channels. Such applications include communications for business reasons and military purposes, and most recently, transactions through the Internet.

A variety of *cryptographic* methods have been developed to facilitate this type of communication. These include encryption/decryption transformations and digital signatures.

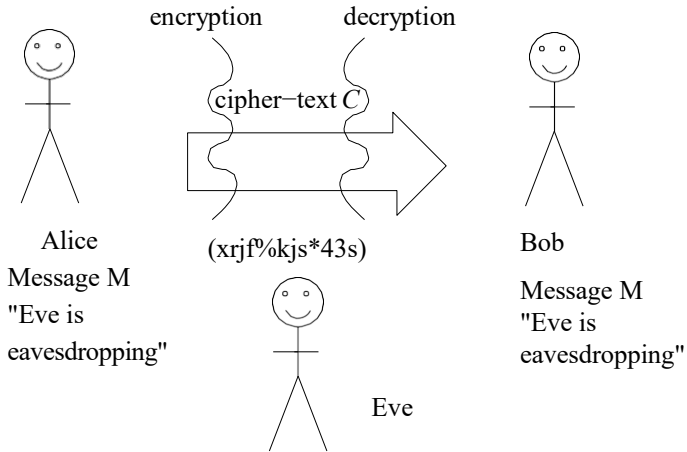
Encryption schemes

Confidentiality in communication can be achieved by *encryption schemes*, or *ciphers*.

The general idea behind these schemes is that the message M to be sent, often referred to as the *plaintext*, is encrypted into an unrecognizable string of characters C , the *ciphertext*.

The ciphertext C is then transmitted to the recipient who decrypts C to recover the original message M .

- **Plaintext (M):** The original message (e.g., "HELLO").
- **Ciphertext (C):** The scrambled secret code (e.g., "KHOOR").
- **Encryption:** Locking the message (plaintext $\rightarrow$ ciphertext).
- **Decryption:** Unlocking the message (ciphertext $\rightarrow$ plaintext).



◆ Scenario:

- Alice wants to send a message (plaintext p) to Bob.
- The communication channel is insecure and can be eavesdropped. If Alice and Bob have previously agreed on an encryption scheme (cipher), the message can be sent encrypted (ciphertext c).

Symmetric encryption schemes and secret keys

In a traditional encryption scheme a common secret key, K , is shared by Alice and Bob. This common key K is used for both encryption and decryption of messages.

Such an encryption scheme is called *symmetric* since the recipient and receiver both have access to the same secret key, and it is used for encryption and decryption.

1. Shared Secret Key:

- Alice and Bob **agree on a key (K)** beforehand (e.g., a password like "7\$gH2pX").

2. Encryption:

- Alice uses **K** to scramble her message (**M**) into ciphertext (**C**).
- Example: "Attack at dawn" + Key **K** → "xY9!q2#v".

3. Decryption:

- Bob uses the **same K** to turn **C** back into **M** .

Symmetric encryption schemes and secret keys

Substitution ciphers

A classic example of a symmetric cipher is a *substitution* cipher. In this case the secret key is a *permutation* π of the characters of the alphabet. (For example, each A gets replaced by the letter D , each B gets replaced by the letter H , etc.)

Encryption of the plaintext M is accomplished by mapping each character x with its corresponding character $y = \pi(x)$.

Decrypting the ciphertext C is easily accomplished with knowledge of the permutation π , i.e. each character y of C is replaced with $x = \pi^{-1}(y)$.

Symmetric encryption schemes and secret keys

The Caesar cipher

The **Caesar cipher** is one of the oldest encryption methods, used by **Julius Caesar** over 2,000 years ago to send secret messages.

Shift Letters by a secret number (Key K): Each letter in the message is replaced by another letter k positions down the alphabet.

$$y = (x + k) \bmod n$$

where n is the size of the alphabet and the integer k , $1 \leq k < n$.

- x = Original letter (A=0, B=1, ..., Z=25).
- k = Secret key (e.g., $k=3$).
- $\bmod 26$ = Wraps around after Z (e.g., $Z + 3 \rightarrow C$).

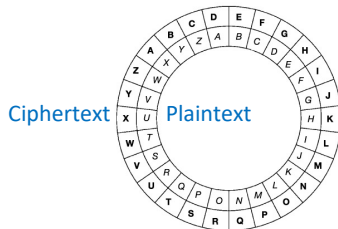
This type of cipher is called the “Caesar cipher” because Julius Caesar is known to have used it with $k = 3$.

Traditional Cryptography

- ◆ Ciphers were already studied in ancient times
- ◆ Caesar's cipher:
 - replace a with d
 - replace b with e
 - ...
 - replace z with c
- ◆ Caesar's cipher is an example of a monoalphabetic substitution cipher, which permutes the characters

$N=3$

Substitution cipher takes the plaintext alphabets and replace them by other alphabets to generate the ciphertext.

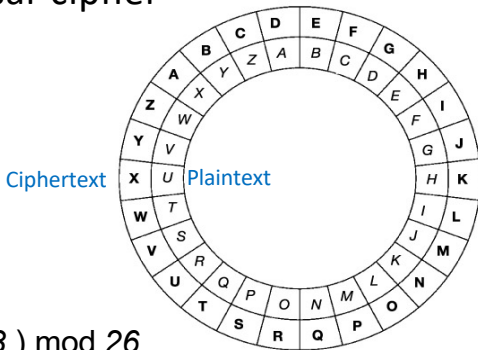


Symmetric encryption schemes and secret keys

The Caesar cipher

Alice

Bob



$$y = (x + 3) \bmod 26$$

$$x = (y - 3) \bmod 26$$

Hello world

Khoor Zruog

Hello world

Substitution cipher takes the plaintext alphabets and replace them by other alphabets to generate the ciphertext.

Symmetric encryption schemes and secret keys

Breaking substitution ciphers

While very easy to use, substitution ciphers are not secure.

The secret key of a substitution cipher is very easily broken by using frequency analysis, based on knowledge of the frequency of the various letters, or groups of letters, in the alphabet being used.

Statistical Attacks

- ◆ Armed with statistical knowledge about the plaintext language, one can easily break a monoalphabetic substitution cipher
 - Most frequent characters in English: e, t, o, a, n, i, ...
 - Most frequent digrams: th, in, er, re, an, ...
 - Most frequent trigrams: the, ing, and, ion, ...

- ◆ The first description of the frequency analysis attack appears in a book written in the 9th century by the Arab philosopher al-Kindi

- ◆ Example (S. Singh, The Code Book, 1999):

PCQ VMJYPD LBYK LYSO KBXBJXWXV BXV ZCJPO EYPD KBXBJYUXJ LBJOO
KCPK. CP LBO LBCMXPV XPV IYJKL PYDBL, QBOP KBO BXV OPVOV LBO
LXRO CI SX'XJMI, KBO JCKO XPV EYKKOV LBO DJCMPV ZOICJO BYS,
KXUYPD: "DJOXL EYPD, ICJ X LBCMXPV XPV CPO PYDBLK Y BXNO ZOOP
JOACMPLYPD LC UCM LBO IXZROK CI FXKL XDOK XPV LBO RODOPVK CI
XPAYOPL EYPDK. SXU Y SXEO KC ZCRV XK LC AJXNO X IXNCMJ CI UCMJ
SXGOKLU?"

OFYRCDMO, LXROK IJCS LBO LBCMXPV XPV CPO PYDBLK

Frequency Analysis (1)

- ◆ We identify the most common characters, digrams and trigrams in the ciphertext

- ◆ Example

PCQ VMJYPD LBYK LYSO KBXBJXWXV BXV ZCJPO EYPD
KBXBJYUXJ LBJOO KCPK. CP LBO LBCMXPV XPV IYJKL PYDBL,
QBOP KBO BXV OPVOV LBO LXRO CI SX'XJMI, KBO JCKO XPV
EYKKOV LBO DJCMPV ZOICJO BYS, KXUYPD: "DJOXL EYPD, ICJ
X LBCMXPV XPV CPO PYDBLK Y BXNO ZOOP JOACMPLYPD LC
UCM LBO IXZROK CI FXKL XDOK XPV LBO RODOPVK CI XPAYOPL
EYDPK. SXU Y SXEO KC ZCRV XK LC AJXNO X IXNCMJ CI UCMJ
SXGOKLU?"

OFYRCDMO, LXROK IJCS LBO LBCMXPV XPV CPO PYDBLK

- ◆ First guess:

- LBO is THE

Frequency Analysis (2)

- Assuming LBO represents THE, we replace L with T, B with H, and O with E and get

PCQ VMJYPD THYK TYSE KHXHJXWXV HXV ZCJPE EYPD
KHXHJYUXJ THJEE KCPK. CP THE THCMKXPV XPV IYJKT
PYDHT, QHEP KHO HXV EPVEV THE LXRE CI SX'XJMI, KHE JCKE
XPV EYKKEV THE DJCMPV ZEICJE HYS, KXUYPD: "DJEXT EYPD,
ICJ X THCMKXPV XPV CPE PYDHTK Y HXNE ZEEP JEACMPTYPD
TC UCM THE IXZREK CI FXKT XDEK XPV THE REDEPVK CI
XPAYEPT EYPAK. SXU Y SXEE KC ZCRV XK TC AJXNE X IXNCMJ
CI UCMJ SXGEKTU?"
EFYRCOME, TXREK IJCS THE THCMKXPV XPV CPE PYDBTK

Symmetric encryption schemes and secret keys

Decryption

◆ Code:

X	Z	A	V	O	I	D	B	Y	G	E	R	S	P	C	F	H	J	K	L	M	N	Q	T	U	W
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z

◆ Ciphertext:

PCQ VMJYPD LBYK LYSO KBXBJXWXV BXV ZCJPO EYPD KBXBJYUXJ
LBJOO KCPK. CP LBO LBCMKXPV XPV IYJKL PYDBL, QBOP KBO BXV
OPVOV LBO LXRO CI SX'XJMI, KBO JCKO XPV EYKKOV LBO DJCMPV
ZOICJO BYS, KXUYPD: "DJOXL EYPD, ICJ X LBCMKXPV XPV CPO PYDBLK
Y BXNO ZOOP JOACMPLYPD LC UCM LBO IXZROK CI FXKL XDOK XPV
LBO RODOPVK CI XPAYOPL EYDK. SXU Y SXEO KC ZCRV XK LC AJXNO
X IXNCMJ CI UCMJ SXGOKLU?"
OFYRCDMO, LXROK IJCS LBO LBCMKXPV XPV CPO PYDBLK

◆ Plaintext:

Now during this time Shahrazad had borne King Shahriyar three sons.
On the thousand and first night, when she had ended the tale of
Ma'aruf, she rose and kissed the ground before him, saying: "Great King,
for a thousand and one nights I have been recounting to you the fables
of past ages and the legends of ancient kings. May I make so bold as to
crave a favour of your majesty?"

Epilogue, Tales from the Thousand and One Nights

Brute force

Secret-Key Encryption

- ◆ A secret-key cipher uses a unique key K to encrypt and decrypt
- ◆ Caesar's generalized cipher uses the modular addition of each character (viewed as an integer) with the key:

$$C[i] = P[i] + K \bmod m$$

$$P[i] = C[i] - K \bmod m$$

- ◆ More secure secret-key encryption schemes have been devised in this century
- ◆ Examples:
 - DES
 - 3DES
 - IDEA
 - BLOWFISH
- ◆ With private-key encryption, a distinct secret key must be established for every pair of parties

The One-time pad

Secure symmetric ciphers do exist!

In fact, the most secure cipher known is the symmetric cipher that's referred to as the *one-time pad*.

For this encryption scheme Alice and Bob share a *random* bit string K that is as long as any message that they are going to send. This key K is the symmetric key that is used for the encryption and decryption process.

The length of the key should be greater than or equal to the message key. The key must be one-time and generated randomly each time.

The One-time pad (encryption)

To encrypt the message M , Alice computes $C = M \oplus K$, where the $\oplus$ symbol denotes the bitwise “exclusive-or” operation.

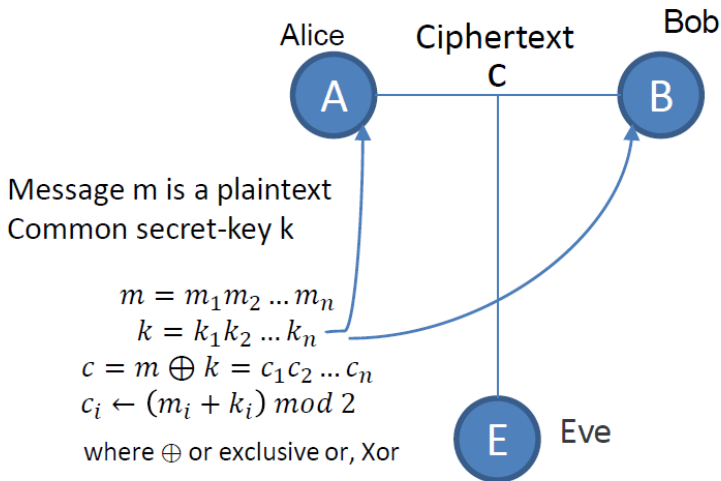
(Note: $0 \oplus 0 = 1 \oplus 1 = 0$ and $0 \oplus 1 = 1 \oplus 0 = 1$.)

Encryption with the one-time pad is based on the exclusive-or (XOR) operation. The exclusive-or of two identical bits (two zeros or two ones) produces a zero and the exclusive-or of two different bits (a zero and a one) produces a one.

- **Key (K):** Random bit string, length $\geq$ plaintext
- **Encrypt:** $C = M \oplus K$ (XOR)
- **Decrypt:** $M = C \oplus K$

Alice then sends C to Bob on any reliable communications channel. The communication is secure because C is computationally indistinguishable from a *random* bit string (This relies highly on the fact that K was selected randomly!).

The One-time pad (encryption)



Example

Encoding letters in binary by ASCII.

Letter	Binary	Letter	Binary
A	01000001	a	01100001
B	01000010	b	01100010
C	01000011	c	01100011
D	01000100	d	01100100
E	01000101	e	01100101
F	01000110	f	01100110
G	01000111	g	01100111
H	01001000	h	01101000
I	01001001	i	01101001
J	01001010	j	01101010
K	01001011	k	01101011
L	01001100	l	01101100
M	01001101	m	01101101
N	01001110	n	01101110
O	01001111	o	01101111
P	01010000	p	01110000

One-time pad

```
1 | 11010111 11100101 10001111 00110000 10100010 00001010 01000000
```

Encryption

```
1 | Plaintext:    01001111 01001110 01000101 01010100 01001001 01001101 01000101
2 | OneTimePad: ^ 11010111 11100101 10001111 00110000 10100010 00001010 01000000
3 | Ciphertext: = 10011000 10101011 11001010 01100100 11101011 01000111 00000101
```

The One-time pad (decryption)

Bob can easily decrypt the ciphertext C to recover M in the following fashion: (Note: $0 \oplus 0 = 1 \oplus 1 = 0$ and $0 \oplus 1 = 1 \oplus 0 = 1$.)

$$C \oplus K = (M \oplus K) \oplus K = M \oplus (K \oplus K) = M \oplus \mathbf{0} = M$$

where $\mathbf{0}$ represents the all-zero string with the same length as M .

This is clearly a symmetric scheme as Alice and Bob use the same key K for encryption and decryption.

The secret pad:
Randomly generated
Not-reused

Letter	Binary	Letter	Binary
A	01000001	a	01100001
B	01000010	b	01100010
C	01000011	c	01100011
D	01000100	d	01100100
E	01000101	e	01100101
F	01000110	f	01100110
G	01000111	g	01100111
H	01001000	h	01101000
I	01001001	i	01101001
J	01001010	j	01101010
K	01001011	k	01101011
L	01001100	l	01101100
M	01001101	m	01101101
N	01001110	n	01101110
O	01001111	o	01101111
P	01010000	p	01110000

One-time pad

```
1 11010111 11100101 10001111 00110000 10100010 00001010 01000000
```

Encryption

```
1 Plaintext: 01001111 01001110 01000101 01010100 01001001 01001101 01000101
2 OneTimePad: ^ 11010111 11100101 10001111 00110000 10100010 00001010 01000000
3 Ciphertext: = 10011000 10101011 11001010 01100100 11101011 01000111 00000101
```

Decryption

```
1 Ciphertext: 10011000 10101011 11001010 01100100 11101011 01000111 00000101
2 OneTimePad: ^ 11010111 11100101 10001111 00110000 10100010 00001010 01000000
3 Plaintext: = 01001111 01001110 01000101 01010100 01001001 01001101 01000101
```

The One-time pad (analysis)

Advantages:

- ▷ Computationally efficient since the bitwise exclusive-or is easy to perform.
- ▷ Very secure (provided K is chosen randomly)

Disadvantages:

- ▷ Alice and Bob must share a very long key K .
- ▷ Security depends on the fact that the key is used *only once*
- In practice, we prefer secret keys that can be reused, and that the keys we use are much shorter than the messages that we must transmit.

How can we do this?

Public-key cryptography

A major problem with symmetric encryption schemes is *key distribution*, or how to *securely* distribute the secret keys.

Another idea is to dispense with using *symmetric* encryption schemes and seek another method for generating (and deciphering) the ciphertexts.

In 1976 Diffie and Hellman described an *abstract* system that overcomes the problem of key distribution – *Public-key cryptosystems*.

Public-key cryptosystems

A public-key cryptosystem consists of an encryption function E and a decryption function D . For any message M , the following properties must hold:

- ▷ $D(E(M)) = M$.
- ▷ Both E and D are easy to compute.
- ▷ It is *computationally infeasible* to derive D from E .
- ▷ $E(D(M)) = M$.

Public-key cryptosystems (cont.)

The third property is the particularly important one. It means that knowledge of the encryption method gives *no information* about the decryption scheme. Anybody can send a private message to the holder of the function D , but only that person knows how to decrypt it.

For this reason E is referred to as a *one-way* function.

In this kind of encryption method E is made *public* and D is kept *private*.



One possible approach

The plaintext M is encrypted using the public keys e and n by the following operation:

$$C \leftarrow M \times e \bmod n.$$

Decryption is again handled by modular multiplication:

$$M \leftarrow C \times d \bmod n.$$

Clearly, if $d \equiv e^{-1} \bmod n$, the scheme can transmit messages.

However, computing modular multiplicative inverse is easy, so this code is not secure and it was never used.

Another possible approach

The plaintext M is encrypted using the public keys e and n by the following operation:

$$C \leftarrow M^e \bmod n.$$

Decryption is again handled by modular exponentiation:

$$M \leftarrow C^d \bmod n.$$

Can we choose e , n , and d , such that for every integer $0 \leq x < n$, we have

$$x^{ed} \equiv x \pmod{n}$$

It should be computationally infeasible to derive d from e and n .

The RSA encryption scheme

Rivest, Shamir, and Adleman proposed a public-key encryption method that is probably the most well-known, and is still in use today for communications via web-browsers, etc.

Their method is tied to the difficulty of *factoring* large numbers. The first step is to select two large *prime* numbers p and q .

Let $n = p \cdot q$ and define $\varphi(n) = (p - 1)(q - 1)$.

We then choose two numbers e and d such that

1. e and $\varphi(n)$ are relatively prime, i.e. $\gcd(e, \varphi(n)) = 1$
2. $ed \equiv 1 \pmod{\varphi(n)}$ (by Extended Euclidean algorithm)

The pair of values e and n form the *public* key and d is the *private* key

Fermat's Little Theorem

Theorem [Fermat's Little Theorem] : Let p be prime, and let x be an integer such that $x \bmod p \neq 0$. Then

$$x^{p-1} \equiv 1 \pmod{p}$$

Fermat's little theorem is a fundamental theorem in elementary number theory, which helps **compute powers of integers modulo prime numbers**.

◆ Example ($p = 5$):

$$1^4 \bmod 5 = 1$$

$$3^4 \bmod 5 = 81 \bmod 5 = 1$$

$$2^4 \bmod 5 = 16 \bmod 5 = 1$$

$$4^4 \bmod 5 = 256 \bmod 5 = 1$$

Fermat's Little Theorem

Theorem [Fermat's Little Theorem] : Let p be prime, and let x be an integer such that $x \bmod p \neq 0$. Then

$$x^{p-1} \equiv 1 \pmod{p}$$

Proof:

We have $\mathbf{Z}_p^* = \{1, 2, \dots, p-1\}$ (numbers from 1 to $p-1$)

Since $\gcd(x, p) = 1$, multiplying all numbers in $\mathbf{Z}_p^*$ by x reorders the set:

$$x\mathbf{Z}_p^* = \{x \cdot 1 \bmod p, x \cdot 2 \bmod p, \dots, x \cdot (p-1) \bmod p\} = \{1, 2, \dots, p-1\}$$

Multiply all numbers in the original set and the reordered set:

$$1 \cdot 2 \cdots (p-1) \equiv (x \cdot 1)(x \cdot 2) \cdots (x \cdot (p-1)) \pmod{p}.$$

$$(p-1)! \equiv x^{p-1} (p-1)! \pmod{p}$$

Due to $\gcd((p-1)!, p) = 1$, so we multiply the multiplicative inverse of $(p-1)!$.

Then we get :

$$x^{p-1} \equiv 1 \pmod{p}$$

Fermat's Little Theorem

Theorem [Fermat's Little Theorem] : Let p be prime, and let x be an integer such that $x \bmod p \neq 0$. Then

$$x^{p-1} \equiv 1 \pmod{p}$$

Corollary

Let p be a prime. For each nonzero residue x of Z_p , the multiplicative inverse of x is $x^{p-2} \bmod p$

Proof

$$xx^{p-2} \bmod p = x^{p-1} \bmod p = 1$$

$$x^{-1} \equiv x^{p-2} \pmod{p}$$

Euler's function

- ◆ The multiplicative group for Z_n , denoted with Z_n^* , is the subset of elements of Z_n relatively prime with n
- ◆ The totient function of n , denoted with $\phi(n)$, is the size of Z_n^*
- ◆ Example

$$Z_{10}^* = \{1, 3, 7, 9\}$$

$$\phi(10) = 4$$

- ◆ If p is prime, we have

$$Z_p^* = \{1, 2, \dots, (p-1)\}$$

$$\phi(p) = p-1$$

The general formula to compute $\phi(n)$

Let the prime factorisation of n is given by

$n = p_1^{e_1} * \dots * p_n^{e_n}$, then $\phi(n) = n * (1-1/p_1) * \dots * (1-1/p_n)$.

Example:

$$15 = 3 * 5, \phi(15) = 15 * (1-1/3) * (1-1/5) = 15 * (2/3) * (4/5) = 8$$

$$4 = 2^2, \phi(4) = 4 * (1-1/2) = 2$$

Euler's function

- ◆ The multiplicative group for Z_n , denoted with Z_n^* , is the subset of elements of Z_n relatively prime with n
- ◆ The totient function of n , denoted with $\phi(n)$, is the size of Z_n^*
- ◆ Example

$$Z_{10}^* = \{1, 3, 7, 9\} \qquad \phi(10) = 4$$

- ◆ If p is prime, we have

$$Z_p^* = \{1, 2, \dots, (p-1)\} \qquad \phi(p) = p-1$$

Theorem [Euler's Theorem] : Let n be a positive integer, and let x be an integer such that $\gcd(x, n) = 1$. Then

$$x^{\phi(n)} \equiv 1 \pmod{n}$$

- ◆ Example ($n = 10$)

$$3^{\phi(10)} \bmod 10 = 3^4 \bmod 10 = 81 \bmod 10 = 1$$

$$7^{\phi(10)} \bmod 10 = 7^4 \bmod 10 = 2401 \bmod 10 = 1$$

$$9^{\phi(10)} \bmod 10 = 9^4 \bmod 10 = 6561 \bmod 10 = 1$$

Euler's function

Theorem [Euler's Theorem] : Let n be a positive integer, and let x be an integer such that $\gcd(x, n) = 1$. Then

$$x^{\phi(n)} \equiv 1 \pmod{n}$$

Proof:

We have $\mathbf{Z}_n^* = \{a_1 = 1 < a_2 < \dots < a_{\phi(n)}\}$ (numbers are relatively prime with n)

Since $\gcd(x, n) = 1$, multiplying every a_i by x modulo n **reorders** the set:

$$\{x \cdot a_1 \bmod n, x \cdot a_2 \bmod n, \dots, x \cdot a_{\phi(n)} \bmod n\} = \{a_1, a_2, \dots, a_{\phi(n)}\}$$

Multiply all a_i in the original set and the reordered set:

$$\prod_{i=1}^{\phi(n)} a_i = \prod_{i=1}^{\phi(n)} x \cdot a_i \bmod n \equiv x^{\phi(n)} \prod_{i=1}^{\phi(n)} a_i \bmod n$$

Due to $\gcd(\prod_{i=1}^{\phi(n)} a_i, n) = 1$, so we multiply both sides by the multiplicative inverse of $\prod_{i=1}^{\phi(n)} a_i$.

Consequently,

$$x^{\phi(n)} \equiv 1 \pmod{n}$$

The RSA encryption scheme

Rivest, Shamir, and Adleman proposed a public-key encryption method that is probably the most well-known, and is still in use today for communications via web- browsers, etc.

Their method is tied to the difficulty of *factoring* large numbers. The first step is to select two large *prime* numbers p and q .

Let $n = p \cdot q$ and define $\varphi(n) = (p - 1)(q - 1)$.

We then choose two numbers e and d such that

1. e and $\varphi(n)$ are relatively prime, i.e. $\gcd(e, \varphi(n)) = 1$
2. $ed \equiv 1 \pmod{\varphi(n)}$ (by Extended Euclidean algorithm)

The pair of values e and n form the *public* key and d is the *private* key

The RSA encryption scheme

RSA:

- Encryption: $C \leftarrow M^e \bmod n$.
- Decryption: $M \leftarrow C^d \bmod n$.
- $x^{ed} \equiv x \pmod{n}$

Let n, k be positive integers. If x is an integer such that $\gcd(x, n) = 1$, then

$$x^{k\phi(n)+1} \equiv x \pmod{n} \quad (\text{from Euler's Theorem})$$

Can we find d, e, k such that $de = k\phi(n) + 1$?

If we further assume $\gcd(e, \phi(n)) = 1$, finding d becomes finding multiplicative inverse of $e \bmod \phi(n)$.

So, we have found a commutative encoding and decoding method.

Now, is it computationally infeasible to derive d from e and n

The RSA encryption scheme

Let n, k be positive integers. If x is an integer such that $\gcd(x, n) = 1$, then

$$x^{k\phi(n)+1} \equiv x \pmod{n}$$

If $\gcd(e, \phi(n)) = 1$, then

$$de = k\phi(n) + 1$$

For what kind of n , finding d given e and n is hard?

We know that given $\phi(n)$, the solution can be found by the extended Euclidean algorithm.

So, we must choose n such that $\phi(n)$ is hard to compute.

Recall that $\phi(n)$ can be computed by factorizing n .

Therefore, we must choose n that is hard to factorize.

The RSA encryption scheme

The first step is to select two large prime numbers p and q .

Let $n = p \times q$ and have $\phi(n) = (p - 1)(q - 1)$.

We then choose two numbers e and d such that

- e and $\phi(n)$ are relatively prime, so that e has a multiplicative inverse modular $\phi(n)$, i.e., $de = k\phi(n) + 1$ has solution.
- $de \equiv 1 \pmod{\phi(n)}$ (by Extended Euclidean algorithm)

The pair of values e and n form the public key and d is the private key

RSA Cryptosystem

◆ Setup:

- $n = pq$, with p and q primes
- e relatively prime to $\phi(n) = (p - 1)(q - 1)$
- d inverse of e in $Z_{\phi(n)}$

◆ Keys:

- Public key: $K_E = (n, e)$
- Private key: $K_D = d$

◆ Encryption:

- Plaintext M in Z_n
- $C = M^e \bmod n$

◆ Decryption:

- $M = C^d \bmod n$

RSA Cryptosystem

◆ Setup:

- $n = pq$, with p and q primes
- e relatively prime to $\phi(n) = (p - 1)(q - 1)$
- d inverse of e in $Z_{\phi(n)}$

◆ Keys:

- Public key: $K_E = (n, e)$
- Private key: $K_D = d$

◆ Encryption:

- Plaintext M in Z_n
- $C = M^e \bmod n$

◆ Decryption:

- $M = C^d \bmod n$

◆ Example

■ Setup:

- ◆ $p = 7, q = 17$
- ◆ $n = 7 \cdot 17 = 119$
- ◆ $\phi(n) = 6 \cdot 16 = 96$
- ◆ $e = 5$
- ◆ $d = 77$

■ Keys:

- ◆ public key: $(119, 5)$
- ◆ private key: 77

■ Encryption:

- ◆ $M = 19$
- ◆ $C = 19^5 \bmod 119 = 66$

■ Decryption:

- ◆ $M = 66^{77} \bmod 119 = 19$

Complete RSA Example

◆ Setup:

- $p = 5, q = 11$
- $n = 5 \cdot 11 = 55$
- $\phi(n) = 4 \cdot 10 = 40$
- $e = 3$
- $d = 27 \text{ (} 3 \cdot 27 = 81 = 2 \cdot 40 + 1 \text{)}$

◆ Encryption

$$\blacksquare C = M^3 \bmod 55$$

◆ Decryption

$$\blacksquare M = C^{27} \bmod 55$$

<i>M</i>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
<i>C</i>	1	8	27	9	15	51	13	17	14	10	11	23	52	49	20	26	18	2
<i>M</i>	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36
<i>C</i>	39	25	21	33	12	19	5	31	48	7	24	50	36	43	22	34	30	16
<i>M</i>	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54
<i>C</i>	53	37	29	35	6	3	32	44	45	41	38	42	4	40	46	28	47	54

Difficulty of breaking RSA

Note that even knowing e doesn't allow us to figure out d , unless we know $\varphi(n)$. As the ability to factor larger numbers increases, we simply have to choose larger primes p and q so that $n = pq$ is outside of the current factoring capabilities.

RSA Factoring Challenge
https://browse.library.kiwix.org/content/wikipedia_en_all_maxi/RSA_Factoring_Challenge

RSA number	Decimal digits	Binary digits	Cash prize offered	Factored on	Factored by
RSA100	100	330	US\$1,000 ^[9]	April 1, 1991 ^[9]	Arjen K. Lenstra
RSA110	110	364	US\$4,429 ^[9]	April 14, 1992 ^[9]	Arjen K. Lenstra and M.S. Manasse
RSA120	120	397	US\$5,896 ^[9]	July 9, 1993 ^[10]	T. Denny <i>et al.</i>
RSA129 ^[9]	129	426	US\$100	April 26, 1994 ^[9]	Arjen K. Lenstra <i>et al.</i>
RSA130	130	430	US\$14,527 ^[9]	April 10, 1996	Arjen K. Lenstra <i>et al.</i>
RSA140	140	463	US\$17,226	February 2, 1999	Herman te Riele <i>et al.</i>
RSA150	150	496		April 16, 2004	Kazumaro Aoki <i>et al.</i>
RSA155	155	512	US\$9,383 ^[9]	August 22, 1999	Herman te Riele <i>et al.</i>
RSA160	160	530		April 1, 2003	Jens Franke <i>et al.</i> , University of Bonn
RSA170 ^[9]	170	563		December 29, 2009	D. Bonebrinker and M. Krone ^[1]
RSA576	174	576	US\$10,000	December 3, 2003	Jens Franke <i>et al.</i> , University of Bonn
RSA180 ^[9]	180	596		May 8, 2010	S. A. Danilov and I. A. Popovyan, Moscow State University ^[11]
RSA190 ^[9]	190	629		November 8, 2010	A. Timofeev and I. A. Popovyan
RSA640	193	640	US\$20,000	November 2, 2005	Jens Franke <i>et al.</i> , University of Bonn
RSA200 ^[9] ?	200	663		May 9, 2005	Jens Franke <i>et al.</i> , University of Bonn
RSA210 ^[9]	210	696		September 26, 2013 ^[12]	Ryan Propper
RSA704 ^[9]	212	704	US\$30,000	July 2, 2012	Shi Bai, Emmanuel Thomé and Paul Zimmermann
RSA220 ^[9]	220	729		May 13, 2016	S. Bai, P. Gaudry, A. Kruppa, E. Thomé and P. Zimmermann
RSA230 ^[9]	230	762		August 15, 2018	Samuel S. Gross, Noblis, Inc.
RSA232 ^[9]	232	768		February 17, 2020 ^[13]	N. L. Zamarashkin, D. A. Zheltkov and S. A. Matveev.
RSA768 ^[9]	232	768	US\$50,000	December 12, 2009	Thorsten Kleinjung <i>et al.</i> ^[14]
RSA240 ^[9]	240	795		Dec 2, 2019 ^[15]	F. Boudot, P. Gaudry, A. Guillemin, N. Heninger, E. Thomé and P. Zimmermann
RSA250 ^[9]	250	829		Feb 28, 2020 ^[16]	F. Boudot, P. Gaudry, A. Guillemin, N. Heninger, E. Thomé and P. Zimmermann
RSA260	260	862			
RSA270	270	895			
RSA896	270	896	US\$75,000 ^[9]		
RSA280	280	928			
RSA290	290	962			
RSA300	300	995			
RSA309	309	1024			
RSA1024	309	1024	US\$100,000 ^[9]		

Modular Exponentiation

A possible bottleneck in the RSA algorithm is computing expressions of the form

$$x^k \bmod n.$$

The “naive approach” is to calculate $x^2 \bmod n$, then use that to get $x^3 \bmod n$, then $x^4 \bmod n$, etc. So the complexity should be in $O(k)$.

Modular Exponentiation

$$3^{94}(\text{mod } 17)$$

Any number can be represented as the sum of distinct powers of two.

$$94=64+16+8+4+2$$

How do you pick them?

94 =	1	0	1	1	1	1	0
	2^6	2^5	2^4	2^3	2^2	2^1	2^0
	$1*64$	$0*32$	$1*16$	$1*8$	$1*4$	$1*2$	$0*1$

Modular Exponentiation

$$3^{94}(\bmod 17)$$

Any number can be represented as the sum of distinct powers of two.

$$94=64+16+8+4+2$$

Use the smallest numbers whether they are positive or negative

$$\begin{aligned}\text{mod } 17 \quad & 3^2 \equiv 9 \\ & 3^4 \equiv 81 \equiv 13 \equiv -4 \\ & 3^8 \equiv (3^4)^2 \equiv 16 \equiv -1 \\ & 3^{16} \equiv (3^8)^2 \equiv (-1)^2 \equiv 1 \\ & 3^{64} \equiv (3^{16})^4 \equiv (1)^4 \equiv 1\end{aligned}$$

Modular Exponentiation

$$3^{94}(\bmod 17)$$

Any number can be represented as the sum of distinct powers of two.

$$\begin{aligned}\text{mod } 17 \quad 3^{94} &\equiv 3^{64+16+8+4+2} \\ &\equiv 3^{64}3^{16}3^83^43^2 \\ &\equiv (1)(1)(-1)(-4)(9) \\ &\equiv 36 \\ &\equiv 2\end{aligned}$$

Fast exponentiation

$$3^{94} \pmod{17}$$

Any number can be represented as the sum of distinct powers of two.

$$94 = 64 + 16 + 8 + 4 + 2$$

94 =	1	0	1	1	1	1	0
	2^6	2^5	2^4	2^3	2^2	2^1	2^0
	$1 \cdot 64$	$0 \cdot 32$	$1 \cdot 16$	$1 \cdot 8$	$1 \cdot 4$	$1 \cdot 2$	$0 \cdot 1$

$$\begin{aligned} 3^{94} &\equiv 3^{64+16+8+4+2} \\ \pmod{17} \quad &\equiv 3^{64} 3^{16} 3^8 3^4 3^2 \\ &\equiv (1)(1)(-1)(-4)(9) \\ &\equiv 36 \equiv 2 \end{aligned}$$

Fast exponentiation

A possible bottleneck in the RSA algorithm is computing expressions of the form

$$x^k \bmod n.$$

We can do much better with an algorithm based on “repeated squaring.” For example, if we wanted to compute x^{16} , we could first find x^2 , then $(x^2)^2 = x^4$, then $(x^4)^2 = x^8$, and finally $(x^8)^2 = x^{16}$. This requires only four multiplications instead of fifteen with the “naive method.” So the complexity should be in $O(\log k)$.

Digital Signatures

RSA cryptosystem supports *digital signatures*. Suppose that Bob sends a message M to Alice and that Alice wants to *verify* that it was Bob who sent it. Bob can create a *signature* using the decryption function applied to M :

$$S \leftarrow M^d \bmod n.$$

Alice verifies the digital signature using the encryption function, that is by checking that

$$M \equiv S^e \pmod{n}.$$

Since only Bob knows the decryption function, this will verify that it was indeed Bob who sent the message. (Of course, *any* person can use the encryption function as well to reconstruct the message M , so this is not a method to *secretly* pass information from Bob to Alice.)